

LAPORAN PRAKTIKUM DEEP LEARNING

**RECURRENT NEURAL NETWORK UNTUK
PEMROSESAN DATA SEKUENSIAL**



Disusun oleh:

HABIBI PUTRA RIZQULLAH

NIM 2411531001

Kelas B

Dosen Pengampu:

Dr. Derisma, S.T., M.T.

**PROGRAM STUDI INFORMATIKA
FAKULTAS TEKNOLOGI INFORMASI
UNIVERSITAS ANDALAS
2026**

DAFTAR ISI

DAFTAR ISI.....	i
DAFTAR GAMBAR	iii
DAFTAR TABEL	iv
BAB I PENDAHULUAN	1
1.1 Latar Belakang	1
1.2 Rumusan Masalah	1
1.3 Tujuan Praktikum.....	2
1.4 Manfaat Praktikum.....	2
BAB II LANDASAN TEORI	3
2.1 Deep Learning dan Data Sekuensial	3
2.2 Recurrent Neural Network	3
2.3 Hidden State pada RNN	3
2.4 Pembentukan Sequence.....	3
2.5 Fungsi Loss dan Metrik Evaluasi	4
2.6 Optimizer Adam dan Proses Pelatihan	4
2.7 Vanishing dan Exploding Gradient	4
2.8 Hidden Size	4
2.9 Deret Sinusoidal dan Cosine	5
BAB III METODOLOGI DAN LANGKAH PRAKTIKUM	6
3.1 Alat dan Bahan	6
3.2 Alur Pengerjaan.....	6
3.3 Praktikum Prediksi Deret Sinusoidal	7
3.3.1 Persiapan Library dan Perangkat.....	7
3.3.2 Pembuatan Data Sinusoidal.....	8
3.3.3 Pembentukan Sequence.....	9
3.3.4 Definisi Model RNN	10
3.3.5 Fungsi Pelatihan dan Evaluasi.....	11
3.3.6 Pelatihan Model Baseline.....	13
3.3.7 Visualisasi Training Loss	14
3.3.8 Prediksi Deret Sinusoidal	15
3.3.9 Eksperimen Hidden Size	16
3.3.10 Perbandingan Hasil Eksperimen	17
3.3.11 Ringkasan Hasil Praktikum	20
3.4 Tugas Praktikum Prediksi Deret Cosine	21
3.4.1 Persiapan Library dan Perangkat.....	21
3.4.2 Pembuatan Data Cosine	22
3.4.3 Pembentukan Sequence.....	23
3.4.4 Definisi Model RNN	24
3.4.5 Fungsi Eksperimen.....	25
3.4.6 Pelatihan Berdasarkan Hidden Size	26
3.4.7 Penyusunan Tabel Hasil.....	27
3.4.8 Perbandingan Training Loss.....	28
3.4.9 Perbandingan Hasil Prediksi	30
3.4.10 Kesimpulan Tugas Praktikum	32
BAB IV HASIL DAN PEMBAHASAN.....	33
4.1 Hasil Praktikum Deret Sinusoidal	33
4.1.1 Hasil Model Baseline	33
4.1.2 Perbandingan Hidden Size	33
4.1.3 Analisis Loss dan Prediksi	34

4.2 Hasil Tugas Praktikum Deret Cosine	34
4.2.1 Hasil Eksperimen	34
4.2.2 Analisis Hasil Prediksi	34
4.3 Pengaruh Hidden Size terhadap Model	35
4.4 Jawaban Pertanyaan Reflektif	35
4.4.1 Cara RNN Mengingat Informasi	35
4.4.2 Vanishing Gradient	35
4.4.3 Penanganan Urutan Panjang	36
BAB V PENUTUP	37
5.1 Kesimpulan	37
DAFTAR PUSTAKA	38

DAFTAR GAMBAR

Gambar 3.1 Import library dan penentuan perangkat.....	7
Gambar 3.2 Visualisasi data deret sinusoidal.....	8
Gambar 3.3 Pembentukan sequence data sinusoidal	9
Gambar 3.4 Definisi model RNN dan pemeriksaan output.....	10
Gambar 3.5 Bagian awal fungsi pelatihan RNN	11
Gambar 3.6 Evaluasi model dan penyimpanan hasil pelatihan	12
Gambar 3.7 Hasil pelatihan model baseline	13
Gambar 3.8 Grafik training loss model baseline	14
Gambar 3.9 Perbandingan data asli dan prediksi sinusoidal	15
Gambar 3.10 Eksperimen hidden size dan tabel hasil	16
Gambar 3.11 Kode visualisasi perbandingan eksperimen.....	17
Gambar 3.12 Grafik loss untuk setiap hidden size	18
Gambar 3.13 Perbandingan prediksi berdasarkan hidden size	19
Gambar 3.14 Ringkasan pemilihan hidden size	20
Gambar 3.15 Import library untuk tugas cosine	21
Gambar 3.16 Visualisasi data deret cosine.....	22
Gambar 3.17 Pembentukan sequence data cosine.....	23
Gambar 3.18 Definisi arsitektur RNN untuk tugas	24
Gambar 3.19 Bagian awal fungsi eksperimen.....	25
Gambar 3.20 Evaluasi dan pengembalian hasil eksperimen	26
Gambar 3.21 Pelatihan model untuk tiga hidden size	26
Gambar 3.22 Pengolahan hasil eksperimen ke DataFrame	27
Gambar 3.23 Tabel hasil eksperimen data cosine	28
Gambar 3.24 Kode visualisasi perbandingan training loss.....	28
Gambar 3.25 Grafik loss pada data cosine	29
Gambar 3.26 Kode visualisasi hasil prediksi	30
Gambar 3.27 Perbandingan prediksi data cosine.....	31
Gambar 3.28 Kesimpulan hasil eksperimen cosine.....	32

DAFTAR TABEL

Tabel 4.1 Hasil eksperimen hidden size pada data sinusoidal.....	33
Tabel 4.2 Hasil eksperimen hidden size pada data cosine.....	34

BAB I

PENDAHULUAN

1.1 Latar Belakang

Data sekuensial memiliki urutan yang memengaruhi makna setiap nilai. Contohnya dapat ditemukan pada data teks, suara, sensor, dan deret waktu. Model jaringan saraf biasa memproses masukan secara terpisah sehingga hubungan antarlangkah tidak selalu tersimpan. Recurrent Neural Network atau RNN dirancang untuk mengatasi kebutuhan tersebut dengan meneruskan hidden state dari satu langkah ke langkah berikutnya [1], [2].

Pada praktikum ini, RNN digunakan untuk mempelajari pola deret sinusoidal. Sepuluh nilai sebelumnya dipakai sebagai masukan untuk memprediksi satu nilai berikutnya. Pengujian kemudian dilanjutkan pada deret cosine sebagai tugas individu. Selain melihat penurunan loss, percobaan juga membandingkan hidden size 16, 32, 64, dan 128 agar pengaruh kapasitas model terhadap ketelitian serta waktu pelatihan dapat diamati.

1.2 Rumusan Masalah

1. Bagaimana menyiapkan data sinusoidal menjadi sequence yang dapat diproses oleh RNN?
2. Bagaimana membangun dan melatih model RNN menggunakan PyTorch untuk memprediksi nilai berikutnya?
3. Bagaimana perubahan hidden size memengaruhi loss, MAE, pola prediksi, dan waktu pelatihan?
4. Bagaimana kinerja RNN ketika pola data diubah dari sinusoidal menjadi cosine?
5. Apa penyebab vanishing gradient dan teknik apa yang dapat digunakan untuk urutan yang panjang?

1.3 Tujuan Praktikum

1. Menjelaskan konsep dasar RNN dan hidden state pada data berurutan.
2. Mengimplementasikan model RNN sederhana menggunakan PyTorch.
3. Melatih model untuk memprediksi deret sinusoidal dan cosine.
4. Mengevaluasi model menggunakan loss, MAE, bentuk kurva prediksi, dan waktu pelatihan.
5. Menganalisis pengaruh hidden size serta permasalahan vanishing gradient.

1.4 Manfaat Praktikum

Praktikum ini membantu memahami aliran data pada RNN melalui contoh yang sederhana dan mudah divisualisasikan. Hasil eksperimen juga memberi gambaran bahwa penambahan jumlah neuron tersembunyi dapat meningkatkan ketelitian, tetapi tetap menambah beban komputasi dan tidak selalu membuat proses pelatihan lebih stabil.

BAB II

LANDASAN TEORI

2.1 Deep Learning dan Data Sekuensial

Deep learning memanfaatkan jaringan saraf dengan beberapa lapisan untuk mempelajari representasi dari data. Pada data sekuensial, posisi dan urutan nilai menjadi bagian penting karena nilai saat ini dapat berhubungan dengan nilai yang muncul sebelumnya. Oleh sebab itu, model yang digunakan perlu memiliki mekanisme untuk membawa informasi antarlangkah [1].

2.2 Recurrent Neural Network

RNN adalah jaringan saraf yang mempunyai hubungan berulang pada proses komputasinya. Keluaran pada suatu langkah dipengaruhi oleh masukan saat itu dan hidden state dari langkah sebelumnya. Implementasi nn.RNN pada PyTorch dapat memakai aktivasi tanh atau ReLU serta menghasilkan output dan hidden state untuk seluruh urutan [6].

2.3 Hidden State pada RNN

Hidden state berfungsi sebagai ringkasan informasi yang telah dilalui oleh model. Saat satu nilai baru masuk, RNN menggabungkan nilai tersebut dengan hidden state sebelumnya. Hasil penggabungan digunakan untuk membentuk hidden state baru. Pada praktikum ini, output pada langkah terakhir diteruskan ke lapisan linear untuk menghasilkan satu nilai prediksi.

2.4 Pembentukan Sequence

Deret sepanjang 1.000 nilai diubah menjadi pasangan masukan dan target menggunakan sliding window. Nilai sequence length ditetapkan sebesar 10. Dengan pengaturan ini, setiap sampel masukan berisi sepuluh nilai berurutan, sedangkan targetnya merupakan nilai ke-11. Proses tersebut menghasilkan 990 pasangan data.

2.5 Fungsi Loss dan Metrik Evaluasi

Mean Squared Error digunakan sebagai fungsi loss karena tugas yang dikerjakan berupa regresi. MSE menghitung rata-rata kuadrat selisih antara nilai prediksi dan target [7]. Mean Absolute Error digunakan sebagai metrik tambahan karena nilainya menunjukkan rata-rata besar kesalahan dalam satuan yang sama dengan data. Nilai MSE dan MAE yang lebih kecil menunjukkan prediksi yang lebih dekat dengan target.

2.6 Optimizer Adam dan Proses Pelatihan

Bobot model diperbarui menggunakan optimizer Adam dengan learning rate 0,01. Adam memanfaatkan estimasi momen pertama dan kedua dari gradien sehingga laju pembaruan dapat menyesuaikan setiap parameter [4][8]. Pada setiap epoch dilakukan forward pass, perhitungan loss, backpropagation, dan pembaruan parameter. Seluruh percobaan dijalankan selama 50 epoch.

2.7 Vanishing dan Exploding Gradient

Pelatihan RNN menggunakan backpropagation through time. Ketika urutan sangat panjang, gradien dapat berulang kali dikalikan dengan nilai yang kecil atau besar. Kondisi ini dapat menyebabkan vanishing gradient atau exploding gradient. Vanishing gradient membuat bagian awal urutan sulit dipelajari, sedangkan exploding gradient membuat pembaruan bobot tidak stabil [3].

2.8 Hidden Size

Hidden size menentukan jumlah neuron pada hidden state. Nilai yang lebih besar memberi ruang representasi yang lebih luas, tetapi menambah jumlah parameter dan waktu komputasi. Hidden size yang terlalu besar juga tidak menjamin loss lebih kecil karena hasil pelatihan tetap dipengaruhi inisialisasi bobot, learning rate, jumlah epoch, dan karakter data.

2.9 Deret Sinusoidal dan Cosine

Sinusoidal dan cosine dipakai karena keduanya mempunyai pola periodik yang teratur. Kedua pola memiliki bentuk yang sama, tetapi berbeda fase. Data ini sesuai untuk latihan awal RNN karena keberhasilan model dapat diamati langsung melalui kesesuaian kurva prediksi dengan kurva asli.

BAB III

METODOLOGI DAN LANGKAH PRAKTIKUM

3.1 Alat dan Bahan

Alat dan bahan yang digunakan dalam praktikum ini adalah sebagai berikut:

1. Laptop sebagai perangkat untuk menjalankan notebook.
2. Google Colaboratory sebagai lingkungan pemrograman.
3. Python sebagai bahasa pemrograman.
4. PyTorch untuk membangun dan melatih model RNN.
5. NumPy untuk membentuk data numerik.
6. Matplotlib untuk menampilkan grafik.
7. Pandas untuk menyusun tabel hasil eksperimen.
8. Notebook Praktikum3_DeepLearning.ipynb sebagai berkas kerja.

3.2 Alur Pengerjaan

Pengerjaan dimulai dengan menetapkan seed dan memilih perangkat komputasi. Data sinusoidal dibuat sebanyak 1.000 titik, diubah menjadi sequence, lalu dimasukkan ke model RNN. Setelah model baseline dilatih, proses yang sama diulang untuk beberapa hidden size. Pada tugas praktikum, pola data diganti menjadi cosine dan eksperimen dilakukan pada hidden size 16, 32, dan 64.

3.3 Praktikum Prediksi Deret Sinusoidal

3.3.1 Persiapan Library dan Perangkat

Library random dan NumPy digunakan untuk mengatur seed, sedangkan PyTorch digunakan untuk membentuk model. Seed 42 ditetapkan agar hasil lebih

konsisten ketika notebook dijalankan kembali. Perangkat dipilih secara otomatis dan pada pelaksanaan ini model berjalan menggunakan CPU.

```
import random
import time

import matplotlib.pyplot as plt
import numpy as np
import pandas as pd
import torch
import torch.nn as nn

SEED = 42
random.seed(SEED)
np.random.seed(SEED)
torch.manual_seed(SEED)
if torch.cuda.is_available():
    torch.cuda.manual_seed_all(SEED)

device = torch.device("cuda" if torch.cuda.is_available() else "cpu")
print(f"Perangkat yang digunakan: {device}")

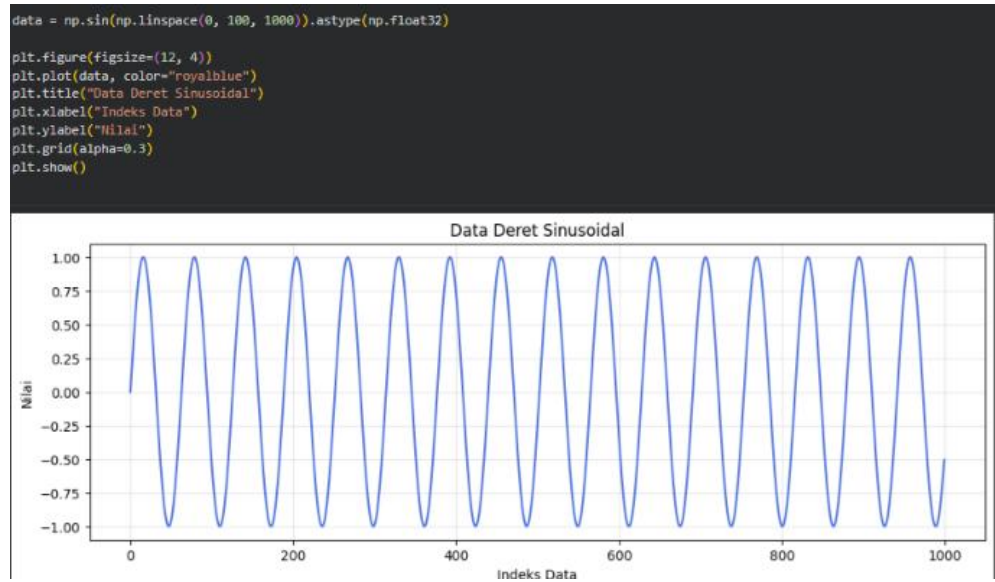
Perangkat yang digunakan: cpu
```

Gambar 3.1 Import library dan penentuan perangkat

3.3.2 Pembuatan Data Sinusoidal

Data dibuat dengan fungsi sinus pada rentang 0 sampai 100 sebanyak 1.000 titik. Grafik menunjukkan pola periodik dengan nilai sekitar -1 sampai 1. Visualisasi

awal diperlukan untuk memastikan bahwa data yang akan dipelajari memiliki pola yang benar.



Gambar 3.2 Visualisasi data deret sinusoidal

3.3.3 Pembentukan Sequence

Fungsi `create_sequences` mengambil sepuluh nilai berurutan sebagai masukan dan satu nilai berikutnya sebagai target. Data kemudian diubah menjadi tensor bertipe

float32 dan ditambahkan dimensi fitur. Bentuk akhir X adalah 990 x 10 x 1, sedangkan y berisi 990 target.

```
seq_length = 10

def create_sequences(series, sequence_length):
    xs, ys = [], []
    for i in range(len(series) - sequence_length):
        xs.append(series[i:i + sequence_length])
        ys.append(series[i + sequence_length])
    return np.array(xs), np.array(ys)

X_np, y_np = create_sequences(data, seq_length)

X = torch.tensor(X_np, dtype=torch.float32).unsqueeze(-1).to(device)
y = torch.tensor(y_np, dtype=torch.float32).unsqueeze(-1).to(device)

print(f"Bentuk X: {X.shape}")
print(f"Bentuk y: {y.shape}")
print(f"Contoh urutan pertama: {X_np[0]}")
print(f"Target pertama: {y_np[0]:.4f}")

*** Bentuk X: torch.Size([990, 10, 1])
    Bentuk y: torch.Size([990, 1])
    Contoh urutan pertama: [0.          0.09993301  0.19886553  0.2958071  0.3897871  0.47986472
    0.56513804  0.64475346  0.7179138  0.7838866 ]
    Target pertama: 0.8420
```

Gambar 3.3 Pembentukan sequence data sinusoidal

3.3.4 Definisi Model RNN

Model terdiri dari satu lapisan nn.RNN dan satu lapisan linear. Hidden size awal ditetapkan sebesar 32. Output pada langkah waktu terakhir dipilih karena bagian tersebut telah membawa informasi dari seluruh sequence yang diproses.

```
class RNNModel(nn.Module):
    def __init__(self, input_size=1, hidden_size=32, output_size=1):
        super().__init__()
        self.rnn = nn.RNN(
            input_size=input_size,
            hidden_size=hidden_size,
            batch_first=True,
        )
        self.fc = nn.Linear(hidden_size, output_size)

    def forward(self, x):
        output, _ = self.rnn(x)
        last_output = output[:, -1, :]
        return self.fc(last_output)

model_check = RNNModel(hidden_size=32).to(device)
dummy_input = torch.randn(4, seq_length, 1, device=device)
print(model_check)
print(f"Bentuk output: {model_check(dummy_input).shape}")

RNNModel(
  (rnn): RNN(1, 32, batch_first=True)
  (fc): Linear(in_features=32, out_features=1, bias=True)
)
Bentuk output: torch.Size([4, 1])
```

Gambar 3.4 Definisi model RNN dan pemeriksaan output

3.3.5 Fungsi Pelatihan dan Evaluasi

Fungsi `train_rnn` dibuat agar pelatihan dapat dipakai kembali pada setiap hidden size. Fungsi menerima data, jumlah epoch, dan learning rate. `MSELoss` digunakan sebagai fungsi loss, sedangkan `Adam` digunakan untuk memperbarui parameter model.

```
def train_rnn(hidden_size, X_data, y_data, epochs=50, learning_rate=0.01, verbose=False):
    torch.manual_seed(SEED)
    if torch.cuda.is_available():
        torch.cuda.manual_seed_all(SEED)

    model = RNNModel(hidden_size=hidden_size).to(device)
    criterion = nn.MSELoss()
    optimizer = torch.optim.Adam(model.parameters(), lr=learning_rate)
    losses = []

    start_time = time.perf_counter()

    for epoch in range(epochs):
        model.train()
        optimizer.zero_grad()

        outputs = model(X_data)
        loss = criterion(outputs, y_data)
        loss.backward()
        optimizer.step()

        losses.append(loss.item())

    if verbose and (epoch + 1) % 10 == 0:
        print(
            f"Epoch [{epoch + 1}/{epochs}] | "
            f"Loss: {loss.item():.6f}"
        )
```

Gambar 3.5 Bagian awal fungsi pelatihan RNN

Setelah seluruh epoch selesai, model dipindahkan ke mode evaluasi. Prediksi akhir, loss, MAE, dan waktu pelatihan dikumpulkan dalam dictionary. Bentuk

penyimpanan ini memudahkan perbandingan beberapa eksperimen tanpa mengulang perhitungan.

```
training_time = time.perf_counter() - start_time

model.eval()
with torch.no_grad():
    predictions = model(X_data)
    final_loss = criterion(predictions, y_data).item()
    mae = torch.mean(torch.abs(predictions - y_data)).item()

return {
    "model": model,
    "losses": losses,
    "predictions": predictions.cpu().numpy().flatten(),
    "final_loss": final_loss,
    "mae": mae,
    "training_time": training_time,
}
```

Gambar 3.6 Evaluasi model dan penyimpanan hasil pelatihan

3.3.6 Pelatihan Model Baseline

Model baseline menggunakan hidden size 32, 50 epoch, dan learning rate 0,01. Loss yang ditampilkan setiap sepuluh epoch turun dari 0,032236 pada epoch ke-10

menjadi 0,000439 pada epoch ke-50. Evaluasi akhir menghasilkan loss 0,000645, MAE 0,022233, dan waktu pelatihan 0,342106 detik.

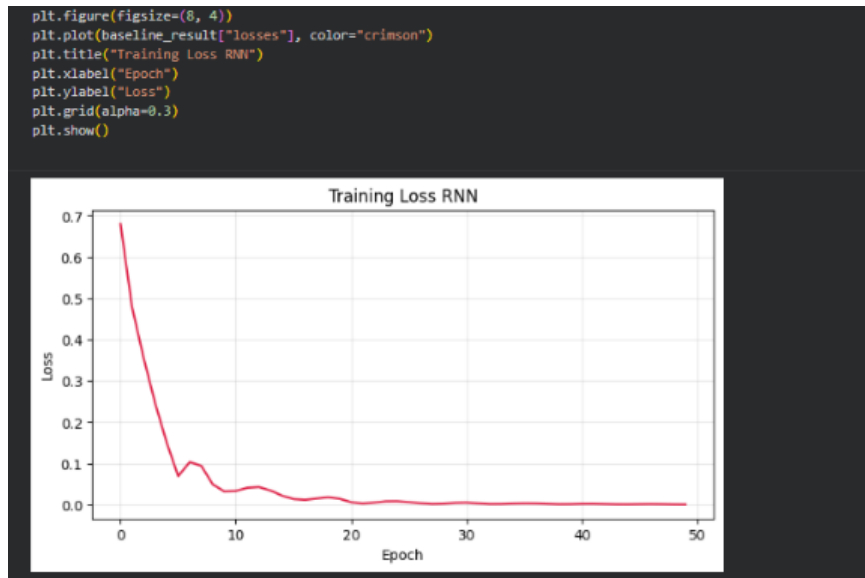
```
baseline_result = train_rnn(  
    hidden_size=32,  
    X_data=X,  
    y_data=y,  
    epochs=50,  
    learning_rate=0.01,  
    verbose=True,  
)  
  
print(f"Loss akhir: {baseline_result['final_loss']:.6f}")  
print(f"MAE: {baseline_result['mae']:.6f}")  
print(f"Waktu training: {baseline_result['training_time']:.2f} detik")  
  
Epoch [10/50] | Loss: 0.032236  
Epoch [20/50] | Loss: 0.014353  
Epoch [30/50] | Loss: 0.004758  
Epoch [40/50] | Loss: 0.001186  
Epoch [50/50] | Loss: 0.000439  
Loss akhir: 0.000645  
MAE: 0.022233  
Waktu training: 0.34 detik
```

Gambar 3.7 Hasil pelatihan model baseline

3.3.7 Visualisasi Training Loss

Grafik training loss memperlihatkan penurunan tajam pada epoch awal, kemudian melandai setelah model mulai mengikuti pola data. Tidak terlihat kenaikan

loss yang menetap. Kondisi tersebut menunjukkan proses optimasi pada model baseline berjalan dengan baik.

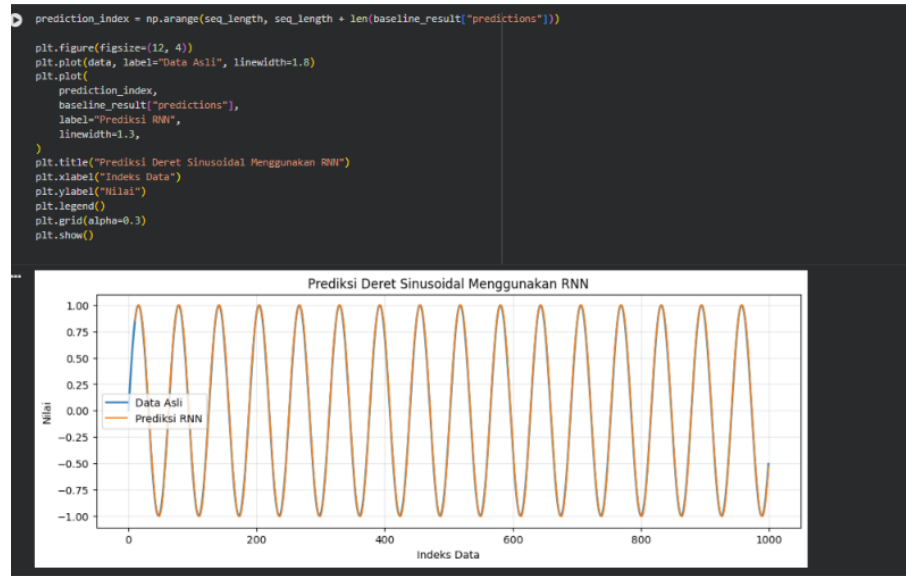


Gambar 3.8 Grafik training loss model baseline

3.3.8 Prediksi Deret Sinusoidal

Prediksi model dibandingkan dengan data asli pada indeks yang sesuai. Garis prediksi hampir menutupi garis data asli pada sebagian besar titik. Selisih kecil masih

terlihat pada bagian awal karena model belum memiliki konteks sebelum sequence pertama.



Gambar 3.9 Perbandingan data asli dan prediksi sinusoidal

3.3.9 Eksperimen Hidden Size

Eksperimen dilanjutkan pada hidden size 16, 64, dan 128, sedangkan hasil baseline 32 tetap digunakan. Setiap model memakai jumlah epoch dan learning rate

yang sama sehingga perbedaan hasil terutama berasal dari kapasitas hidden state dan proses optimasinya.

```
experiment_results = (32: baseline_result)

for hidden_size in [16, 64, 128]:
    print(f"Melatih RNN dengan hidden size {hidden_size}...")
    experiment_results[hidden_size] = train_rnn(
        hidden_size=hidden_size,
        X_data=X,
        y_data=y,
        epochs=50,
        learning_rate=0.01,
    )

comparison_rows = []
for hidden_size in sorted(experiment_results):
    result = experiment_results[hidden_size]
    comparison_rows.append([
        "Hidden Size": hidden_size,
        "Loss Akhir": result["final_loss"],
        "MAE": result["mae"],
        "Waktu Training (s)": result["training_time"],
    ])

comparison_table = pd.DataFrame(comparison_rows)
comparison_table[["Loss Akhir", "MAE", "Waktu Training (s)"]] = (
    comparison_table[["Loss Akhir", "MAE", "Waktu Training (s)"]].round(6)
)
comparison_table
```

```
--- Melatih RNN dengan hidden size 16...
Melatih RNN dengan hidden size 64...
Melatih RNN dengan hidden size 128...
  Hidden Size  Loss Akhir    MAE  Waktu Training (s)
0          16    0.001317  0.030857         0.170915
1          32    0.000645  0.022233         0.342106
2          64    0.000309  0.015652         0.631057
3         128    0.017990  0.116396         1.679071
```

Gambar 3.10 Eksperimen hidden size dan tabel hasil

3.3.10 Perbandingan Hasil Eksperimen

Kode berikut menampilkan loss setiap eksperimen dan membandingkan prediksi yang dihasilkan. Pengurutan hidden size dilakukan agar grafik dapat dibaca mulai dari model yang paling kecil sampai model yang paling besar.

```
plt.figure(figsize=(10, 5))
for hidden_size in sorted(experiment_results):
    plt.plot(
        experiment_results[hidden_size]["losses"],
        label=f"Hidden Size {hidden_size}",
    )
plt.title("Perbandingan Loss Setiap Hidden Size")
plt.xlabel("Epoch")
plt.ylabel("Loss")
plt.legend()
plt.grid(alpha=0.3)
plt.show()

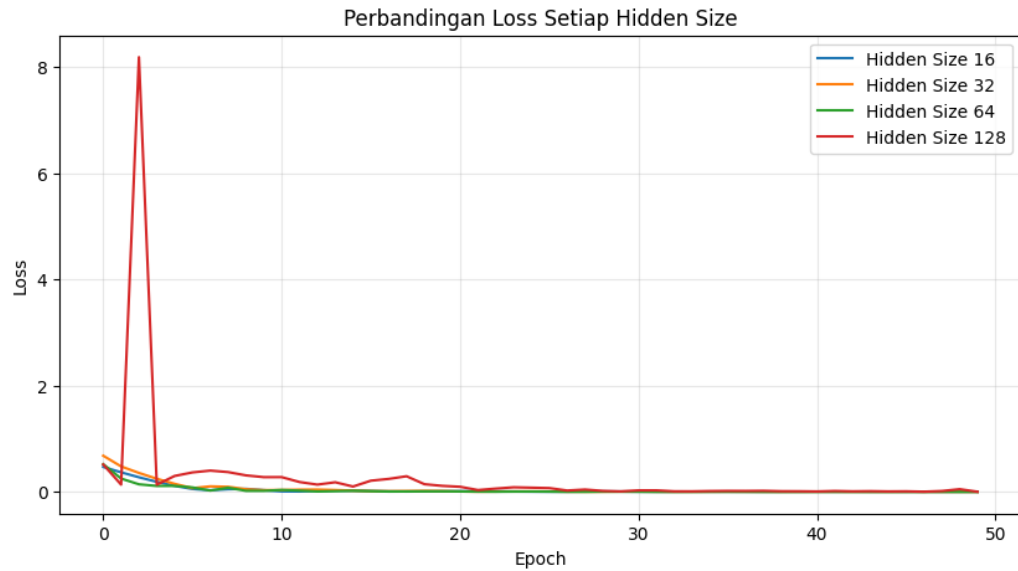
fig, axes = plt.subplots(2, 2, figsize=(14, 8), sharex=True, sharey=True)
for ax, hidden_size in zip(axes.flat, sorted(experiment_results)):
    ax.plot(data, label="Asli", linewidth=1.5)
    ax.plot(
        prediction_index,
        experiment_results[hidden_size]["predictions"],
        label="Prediksi",
        linewidth=1.1,
    )
    ax.set_title(f"Hidden Size {hidden_size}")
    ax.grid(alpha=0.3)
    ax.legend()

fig.suptitle("Perbandingan Prediksi Deret Sinusoidal", fontsize=14)
plt.tight_layout()
plt.show()
```

Gambar 3.11 Kode visualisasi perbandingan eksperimen

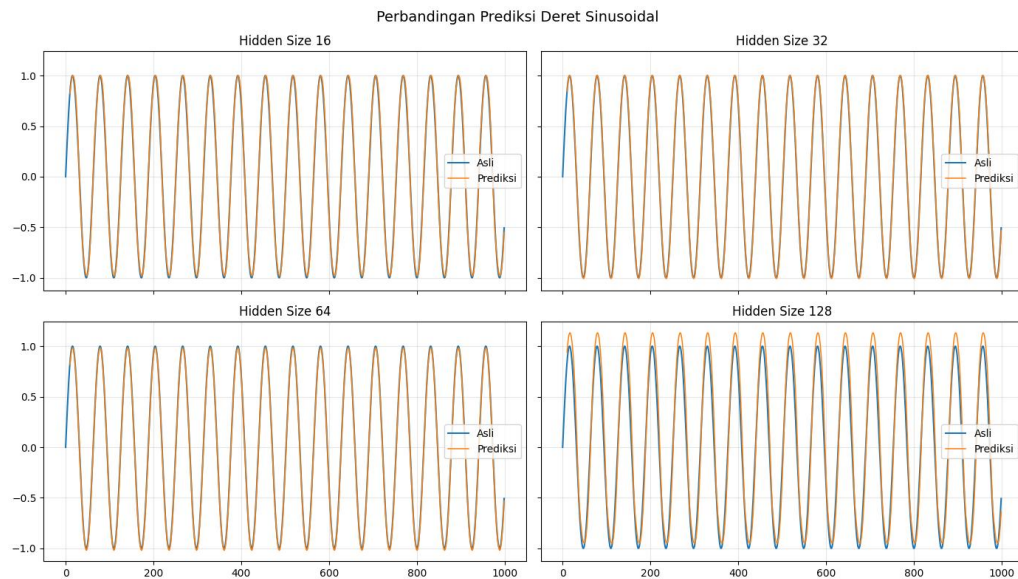
Hidden size 16, 32, dan 64 menunjukkan loss yang turun mendekati nol. Hidden size 128 sempat mengalami lonjakan dan penurunan yang kurang stabil. Hasil ini

memperlihatkan bahwa kapasitas yang lebih besar tidak selalu langsung memberikan optimasi yang lebih baik pada pengaturan yang sama.



Gambar 3.12 Grafik loss untuk setiap hidden size

Perbandingan kurva prediksi menunjukkan hidden size 64 mengikuti pola asli paling rapat. Hidden size 16 dan 32 juga menghasilkan bentuk yang sesuai, sedangkan hidden size 128 masih menampilkan penyimpangan yang lebih terlihat.



Gambar 3.13 Perbandingan prediksi berdasarkan hidden size

3.3.11 Ringkasan Hasil Praktikum

Ringkasan program memilih hidden size 64 sebagai model dengan loss terendah, yaitu 0,000309. Hidden size 16 menjadi model tercepat dengan waktu 0,170915 detik. Pemilihan model akhir tetap perlu menyesuaikan kebutuhan antara ketelitian dan waktu komputasi.

```
best_row = comparison_table.loc[comparison_table["Loss Akhir"].idxmin()]
fastest_row = comparison_table.loc[comparison_table["Waktu Training (s)"].idxmin()]

print("Ringkasan hasil:")
print(
    f"- Loss terendah diperoleh hidden size {int(best_row['Hidden Size'])} "
    f"dengan loss {best_row['Loss Akhir']:.6f}."
)
print(
    f"- Waktu tercepat diperoleh hidden size {int(fastest_row['Hidden Size'])} "
    f"selama {fastest_row['Waktu Training (s)']:.6f} detik."
)
print("- Pemilihan model perlu mempertimbangkan ketelitian dan waktu training.")

*** Ringkasan hasil:
- Loss terendah diperoleh hidden size 64 dengan loss 0.000309.
- Waktu tercepat diperoleh hidden size 16 selama 0.170915 detik.
- Pemilihan model perlu mempertimbangkan ketelitian dan waktu training.
```

Gambar 3.14 Ringkasan pemilihan hidden size

3.4 Tugas Praktikum Prediksi Deret Cosine

3.4.1 Persiapan Library dan Perangkat

Bagian tugas menggunakan library yang sama dan menambahkan Pandas untuk menyusun hasil percobaan dalam bentuk tabel. Seed tetap bernilai 42 agar perbandingan antarmodel dilakukan dengan kondisi awal yang konsisten.

```
import random
import time

import matplotlib.pyplot as plt
import numpy as np
import pandas as pd
import torch
import torch.nn as nn

SEED = 42
random.seed(SEED)
np.random.seed(SEED)
torch.manual_seed(SEED)
if torch.cuda.is_available():
    torch.cuda.manual_seed_all(SEED)

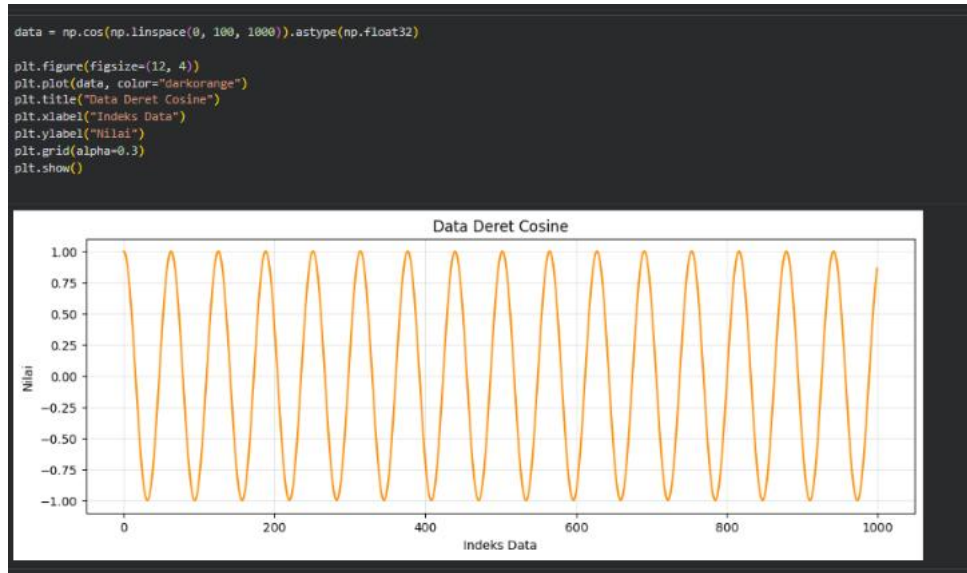
device = torch.device("cuda" if torch.cuda.is_available() else "cpu")
print(f"Perangkat yang digunakan: {device}")

*** Perangkat yang digunakan: cpu
```

Gambar 3.15 Import library untuk tugas cosine

3.4.2 Pembuatan Data Cosine

Data tugas dibuat dengan fungsi cosine pada rentang dan jumlah titik yang sama dengan praktikum. Grafik memastikan pola cosine terbentuk dengan benar sebelum data dimasukkan ke tahap pembentukan sequence.



Gambar 3.16 Visualisasi data deret cosine

3.4.3 Pembentukan Sequence

Data cosine dipotong menjadi sequence sepanjang 10. Hasilnya sama-sama menghasilkan X dengan bentuk 990 x 10 x 1 dan y dengan bentuk 990 x 1. Perbedaannya hanya terletak pada pola nilai yang digunakan.

```
seq_length = 10

def create_sequences(series, sequence_length):
    xs, ys = [], []
    for i in range(len(series) - sequence_length):
        xs.append(series[i:i + sequence_length])
        ys.append(series[i + sequence_length])
    return np.array(xs), np.array(ys)

X_np, y_np = create_sequences(data, seq_length)

X = torch.tensor(X_np, dtype=torch.float32).unsqueeze(-1).to(device)
y = torch.tensor(y_np, dtype=torch.float32).unsqueeze(-1).to(device)

print(f"Bentuk X: {X.shape}")
print(f"Bentuk y: {y.shape}")

... Bentuk X: torch.Size([990, 10, 1])
    Bentuk y: torch.Size([990, 1])
```

Gambar 3.17 Pembentukan sequence data cosine

3.4.4 Definisi Model RNN

Arsitektur model tetap terdiri dari nn.RNN dan lapisan linear. Parameter hidden_size dibuat fleksibel karena tugas membandingkan tiga jumlah neuron

tersembunyi. Output pada langkah terakhir kembali digunakan untuk menghasilkan prediksi satu nilai.

```
class RNNModel(nn.Module):
    def __init__(self, input_size=1, hidden_size=32, output_size=1):
        super().__init__()
        self.rnn = nn.RNN(
            input_size=input_size,
            hidden_size=hidden_size,
            batch_first=True,
        )
        self.fc = nn.Linear(hidden_size, output_size)

    def forward(self, x):
        output, _ = self.rnn(x)
        return self.fc(output[:, -1, :])
```

Gambar 3.18 Definisi arsitektur RNN untuk tugas

3.4.5 Fungsi Eksperimen

Fungsi `run_experiment` melatih satu model berdasarkan hidden size yang diberikan. Seed diatur ulang pada awal fungsi agar setiap percobaan dimulai secara konsisten. Waktu pelatihan diukur dari awal sampai seluruh epoch selesai.

```
def run_experiment(hidden_size, epochs=50, learning_rate=0.01):
    torch.manual_seed(SEED)
    if torch.cuda.is_available():
        torch.cuda.manual_seed_all(SEED)

    model = RNNModel(hidden_size=hidden_size).to(device)
    criterion = nn.MSELoss()
    optimizer = torch.optim.Adam(model.parameters(), lr=learning_rate)
    losses = []

    start_time = time.perf_counter()

    for epoch in range(epochs):
        model.train()
        optimizer.zero_grad()

        outputs = model(X)
        loss = criterion(outputs, y)
        loss.backward()
        optimizer.step()

        losses.append(loss.item())

    training_time = time.perf_counter() - start_time
```

Gambar 3.19 Bagian awal fungsi eksperimen

Setelah pelatihan, fungsi menghitung prediksi, loss akhir, dan MAE. Seluruh nilai dikembalikan bersama model dan riwayat loss. Data ini kemudian digunakan untuk membuat tabel serta grafik perbandingan.

```
model.eval()
with torch.no_grad():
    predictions = model(X)
    final_loss = criterion(predictions, y).item()
    mae = torch.mean(torch.abs(predictions - y)).item()

return {
    "model": model,
    "losses": losses,
    "predictions": predictions.cpu().numpy().flatten(),
    "final_loss": final_loss,
    "mae": mae,
    "training_time": training_time,
}
```

Gambar 3.20 Evaluasi dan pengembalian hasil eksperimen

3.4.6 Pelatihan Berdasarkan Hidden Size

Percobaan dilakukan untuk hidden size 16, 32, dan 64. Program menampilkan proses pelatihan setiap model dan menyimpan hasilnya dalam dictionary. Ketiga model menggunakan 50 epoch serta learning rate 0,01.

```
hidden_sizes = [16, 32, 64]
results = {}

for hidden_size in hidden_sizes:
    print(f"Melatih model dengan hidden size {hidden_size}...")
    results[hidden_size] = run_experiment(hidden_size=hidden_size)

print("Seluruh percobaan selesai.")

*** Melatih model dengan hidden size 16...
    Melatih model dengan hidden size 32...
    Melatih model dengan hidden size 64...
    Seluruh percobaan selesai.
```

Gambar 3.21 Pelatihan model untuk tiga hidden size

3.4.7 Penyusunan Tabel Hasil

Hasil percobaan dipindahkan ke DataFrame. Pola prediksi dan catatan waktu ditentukan berdasarkan urutan loss serta waktu pelatihan. Nilai numerik dibulatkan sampai enam angka di belakang koma agar perbedaan kecil tetap terlihat.

```
rows = []
for hidden_size in hidden_sizes:
    result = results[hidden_size]
    rows.append({
        "Hidden Size": hidden_size,
        "Loss Akhir": result["final_loss"],
        "MAE": result["mae"],
        "Waktu Training (s)": result["training_time"],
    })

result_table = pd.DataFrame(rows)

loss_order = result_table["Loss Akhir"].rank(method="dense").astype(int)
time_order = result_table["Waktu Training (s)"].rank(method="dense").astype(int)

pola_labels = {
    1: "sangat halus",
    2: "halus",
    3: "agak kasar",
}

time_labels = {
    1: "training paling cepat",
    2: "waktu training sedang",
    3: "training paling lama",
}

result_table["Pola Prediksi"] = loss_order.map(pola_labels)
result_table["Catatan"] = time_order.map(time_labels)

result_table[["Loss Akhir", "MAE", "Waktu Training (s)"]] = (
    result_table[["Loss Akhir", "MAE", "Waktu Training (s)"]].round(6)
)
```

Gambar 3.22 Pengolahan hasil eksperimen ke DataFrame

Tabel menunjukkan penurunan loss dan MAE ketika hidden size bertambah. Pada saat yang sama, waktu pelatihan meningkat. Hidden size 64 memperoleh loss terendah dan pola paling halus, sedangkan hidden size 16 menjadi yang tercepat.

```

result_table = result_table[
    [
        "Hidden Size",
        "Loss Akhir",
        "Pola Prediksi",
        "Catatan",
        "MAE",
        "Waktu Training (s)",
    ]
]
result_table

```

	Hidden Size	Loss Akhir	Pola Prediksi	Catatan	MAE	Waktu Training (s)
0	16	0.001187	agak kasar	training paling cepat	0.029918	0.172684
1	32	0.000489	halus	waktu training sedang	0.019445	0.263912
2	64	0.000228	sangat halus	training paling lama	0.012119	0.591929

Gambar 3.23 Tabel hasil eksperimen data cosine

3.4.8 Perbandingan Training Loss

Riwayat loss setiap hidden size digambar dalam satu grafik agar kecepatan konvergensi dapat dibandingkan. Label dan warna yang berbeda digunakan untuk membedakan setiap model.

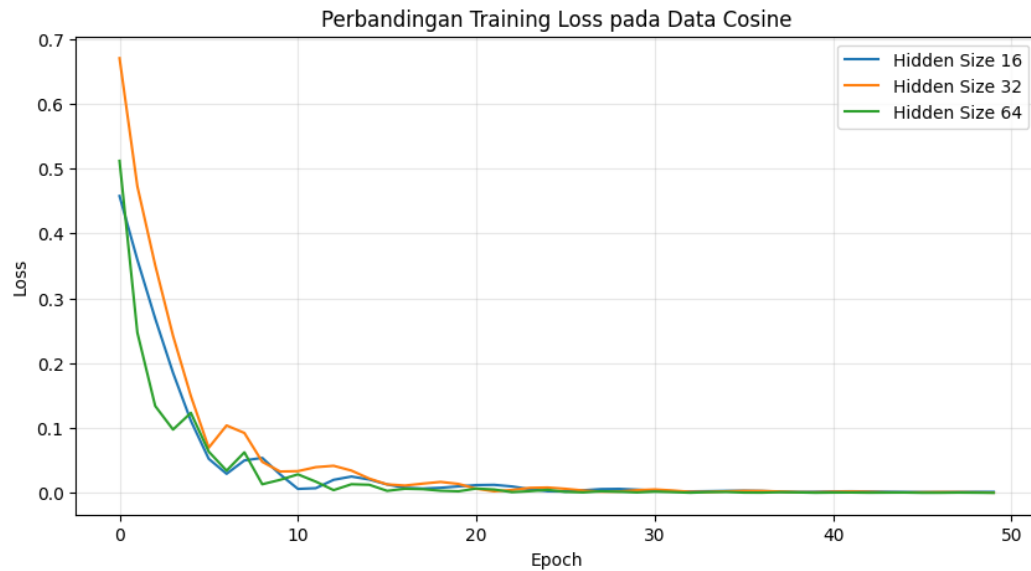
```

plt.figure(figsize=(10, 5))
for hidden_size in hidden_sizes:
    plt.plot(
        results[hidden_size][["losses"]],
        label=f"Hidden Size {hidden_size}",
    )
plt.title("Perbandingan Training Loss pada Data Cosine")
plt.xlabel("Epoch")
plt.ylabel("Loss")
plt.legend()
plt.grid(alpha=0.3)
plt.show()

```

Gambar 3.24 Kode visualisasi perbandingan training loss

Ketiga kurva turun mendekati nol sebelum epoch ke-20. Hidden size 64 mencapai loss yang paling rendah dan menunjukkan penurunan yang relatif cepat. Hidden size 16 memiliki sisa loss sedikit lebih besar dibandingkan model lain.



Gambar 3.25 Grafik loss pada data cosine

3.4.9 Perbandingan Hasil Prediksi

Prediksi setiap model ditampilkan pada subplot yang terpisah. Susunan ini memudahkan pemeriksaan bentuk kurva pada hidden size 16, 32, dan 64 tanpa menumpuk terlalu banyak garis dalam satu grafik.

```
prediction_index = np.arange(seq_length, seq_length + len(y_np))

fig, axes = plt.subplots(3, 1, figsize=(13, 11), sharex=True, sharey=True)

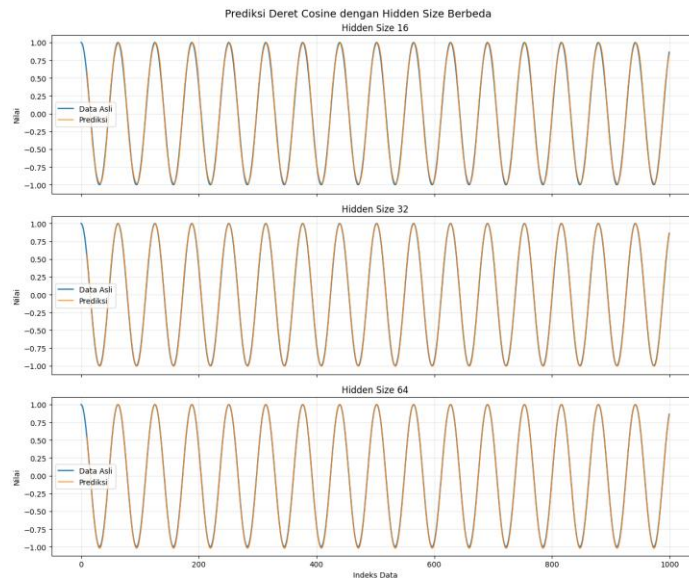
for ax, hidden_size in zip(axes, hidden_sizes):
    ax.plot(data, label="Data Asli", linewidth=1.7)
    ax.plot(
        prediction_index,
        results[hidden_size]["predictions"],
        label="Prediksi",
        linewidth=1.2,
    )
    ax.set_title(f"Hidden Size {hidden_size}")
    ax.set_ylabel("Nilai")
    ax.grid(alpha=0.3)
    ax.legend()

axes[-1].set_xlabel("Indeks Data")
fig.suptitle("Prediksi Deret Cosine dengan Hidden Size Berbeda", fontsize=14)
plt.tight_layout()
plt.show()
```

Gambar 3.26 Kode visualisasi hasil prediksi

Semua model berhasil mengikuti pola cosine. Prediksi hidden size 64 paling dekat dengan data asli, terutama pada bagian puncak dan lembah. Perbedaannya

memang kecil secara visual, tetapi didukung oleh nilai loss dan MAE yang lebih rendah.



Gambar 3.27 Perbandingan prediksi data cosine

3.4.10 Kesimpulan Tugas Praktikum

Program menyimpulkan hidden size 64 sebagai model paling teliti dengan loss 0,000228. Hidden size 16 merupakan model tercepat dengan waktu 0,172684 detik. Hasil ini menegaskan adanya pertukaran antara ketelitian dan efisiensi.

```
best_model = result_table.loc[result_table["Loss Akhir"].idxmin()]
fastest_model = result_table.loc[result_table["Waktu Training (s)"].idxmin()]

print("Kesimpulan hasil eksperimen:")
print(
    f"1. Hidden size {int(best_model['Hidden Size'])} menghasilkan loss "
    f"terendah sebesar {best_model['Loss Akhir']:.6f}."
)
print(
    f"2. Hidden size {int(fastest_model['Hidden Size'])} memiliki waktu "
    f"training paling cepat, yaitu {fastest_model['Waktu Training (s)']:.6f} detik."
)
print(
    "3. Hidden size yang lebih besar menambah kapasitas model, tetapi "
    "tidak selalu menjadi pilihan paling efisien."
)

--- Kesimpulan hasil eksperimen:
1. Hidden size 64 menghasilkan loss terendah sebesar 0.000228.
2. Hidden size 16 memiliki waktu training paling cepat, yaitu 0.172684 detik.
3. Hidden size yang lebih besar menambah kapasitas model, tetapi tidak selalu menjadi pilihan paling efisien.
```

Gambar 3.28 Kesimpulan hasil eksperimen cosine

BAB IV

HASIL DAN PEMBAHASAN

4.1 Hasil Praktikum Deret Sinusoidal

4.1.1 Hasil Model Baseline

Model baseline dengan hidden size 32 menghasilkan loss akhir 0,000645 dan MAE 0,022233. Training loss turun secara konsisten, sedangkan kurva prediksi hampir berhimpit dengan data sinusoidal asli. Waktu pelatihannya sebesar 0,342106 detik pada CPU.

4.1.2 Perbandingan Hidden Size

Hidden Size	Loss Akhir	MAE	Waktu Training (s)	Catatan
16	0,001317	0,030857	0,170915	Paling cepat
32	0,000645	0,022233	0,342106	Model baseline
64	0,000309	0,015652	0,631057	Loss terendah
128	0,017990	0,116396	1,679071	Pelatihan kurang stabil

Tabel 4.1 Hasil eksperimen hidden size pada data sinusoidal

Hidden size 64 memberikan hasil terbaik pada data sinusoidal dengan loss 0,000309 dan MAE 0,015652. Dibandingkan baseline, loss berkurang sekitar 52 persen, tetapi waktu pelatihan bertambah sekitar 84 persen. Hidden size 16 lebih cepat, namun tingkat kesalahannya lebih tinggi.

4.1.3 Analisis Loss dan Prediksi

Penurunan loss memperlihatkan bahwa model berhasil mempelajari hubungan antara sepuluh nilai masukan dan nilai berikutnya. Prediksi yang hampir berhimpit dengan data asli menandakan bahwa pola periodik berhasil ditangkap. Hidden size 128 justru menghasilkan loss dan MAE yang lebih besar. Hal ini dapat terjadi karena kapasitas model meningkat, tetapi jumlah epoch dan learning rate tidak disesuaikan untuk model yang lebih besar.

4.2 Hasil Tugas Praktikum Deret Cosine

4.2.1 Hasil Eksperimen

Hidden Size	Loss Akhir	Pola Prediksi	MAE	Waktu Training (s)	Catatan
16	0,001187	Agak kasar	0,029918	0,172684	Training paling cepat
32	0,000489	Halus	0,019445	0,263912	Training sedang
64	0,000228	Sangat halus	0,012119	0,591929	Training paling lama

Tabel 4.2 Hasil eksperimen hidden size pada data cosine

Pada data cosine, peningkatan hidden size dari 16 menjadi 64 menurunkan loss dari 0,001187 menjadi 0,000228. MAE juga turun dari 0,029918 menjadi 0,012119. Namun, waktu pelatihan meningkat dari 0,172684 detik menjadi 0,591929 detik.

4.2.2 Analisis Hasil Prediksi

Ketiga model mampu mengikuti bentuk dasar cosine. Hidden size 16 masih memiliki selisih yang lebih terlihat pada beberapa titik, sehingga pola diberi catatan agak kasar. Hidden size 32 menghasilkan kurva yang lebih halus, sedangkan hidden size 64 menjadi yang paling dekat dengan data asli. Tidak ditemukan tanda ketidakstabilan seperti pada hidden size 128 di percobaan sinusoidal.

4.3 Pengaruh Hidden Size terhadap Model

Hasil kedua percobaan menunjukkan bahwa penambahan hidden size sampai 64 meningkatkan kemampuan model dalam mengikuti pola periodik. Perbaikan tersebut terlihat pada loss dan MAE yang menurun. Biaya yang harus dibayar adalah waktu pelatihan yang lebih lama karena jumlah operasi dan parameter bertambah.

Penambahan hidden size tidak dapat dianggap selalu lebih baik. Pada data sinusoidal, hidden size 128 menghasilkan loss 0,017990 dan MAE 0,116396, jauh lebih buruk daripada hidden size 64. Dengan pengaturan praktikum ini, hidden size 64 menjadi pilihan paling teliti, sedangkan hidden size 16 lebih sesuai ketika kecepatan menjadi prioritas.

Istilah akurasi tidak digunakan karena keluaran model berupa nilai kontinu, bukan kelas. Evaluasi yang lebih tepat adalah loss MSE, MAE, kesesuaian kurva prediksi, dan waktu pelatihan. Pemilihan metrik yang sesuai membuat penilaian model tidak menyesatkan.

4.4 Jawaban Pertanyaan Reflektif

4.4.1 Cara RNN Mengingat Informasi

RNN dapat mengingat informasi karena hidden state diteruskan dari satu langkah waktu ke langkah berikutnya. Hidden state yang baru dibentuk dari masukan saat ini dan hidden state sebelumnya. Dengan cara tersebut, keluaran pada langkah terakhir membawa ringkasan informasi dari nilai yang telah diproses.

4.4.2 Vanishing Gradient

Vanishing gradient adalah kondisi ketika gradien mengecil selama proses backpropagation melewati banyak langkah waktu. Bobot yang berhubungan dengan bagian awal urutan menerima pembaruan yang sangat kecil. Dampaknya, RNN sulit mempelajari hubungan yang jaraknya panjang. Sebaliknya, exploding gradient terjadi ketika gradien membesar dan membuat proses pelatihan tidak stabil [3].

4.4.3 Penanganan Urutan Panjang

Untuk urutan yang panjang, RNN sederhana dapat diganti dengan Long Short-Term Memory atau Gated Recurrent Unit. Kedua arsitektur tersebut memiliki mekanisme gerbang yang membantu mempertahankan informasi penting. Gradient clipping dapat digunakan untuk membatasi exploding gradient, sedangkan truncated backpropagation through time dapat membatasi panjang aliran gradien saat pelatihan.

BAB V

PENUTUP

5.1 Kesimpulan

RNN berhasil digunakan untuk memprediksi deret sinusoidal dan cosine dengan sequence length 10. Model baseline hidden size 32 memperoleh loss 0,000645 dan MAE 0,022233 pada data sinusoidal. Hasil terbaik pada praktikum diperoleh hidden size 64 dengan loss 0,000309 dan MAE 0,015652.

Pada tugas cosine, hidden size 64 kembali memberikan loss terendah sebesar 0,000228 dan MAE 0,012119. Hidden size yang lebih besar sampai batas tertentu membuat prediksi lebih halus, tetapi waktu pelatihan meningkat. Percobaan hidden size 128 menunjukkan bahwa kapasitas yang lebih besar tidak selalu menghasilkan model yang lebih baik apabila parameter pelatihan tidak disesuaikan.

DAFTAR PUSTAKA

- [1] Goodfellow, I., Bengio, Y., dan Courville, A. 2016. Deep Learning. Cambridge: MIT Press.
- [2] Elman, J. L. 1990. Finding Structure in Time. *Cognitive Science*, 14(2), 179-211. https://doi.org/10.1207/s15516709cog1402_1.
- [3] Hochreiter, S. 1998. The Vanishing Gradient Problem During Learning Recurrent Neural Nets and Problem Solutions. *International Journal of Uncertainty, Fuzziness and Knowledge-Based Systems*, 6(2), 107-116. <https://doi.org/10.1142/S0218488598000094>.
- [4] Kingma, D. P., dan Ba, J. 2015. Adam: A Method for Stochastic Optimization. *International Conference on Learning Representations*. arXiv:1412.6980.
- [5] Paszke, A., et al. 2019. PyTorch: An Imperative Style, High-Performance Deep Learning Library. *Advances in Neural Information Processing Systems*, 32.
- [6] PyTorch Contributors. 2026. RNN. <https://docs.pytorch.org/docs/stable/generated/torch.nn.RNN.html>. Diakses 25 September 2026.
- [7] PyTorch Contributors. 2026. MSELoss. <https://docs.pytorch.org/docs/stable/generated/torch.nn.MSELoss.html>. Diakses 25 September 2026.
- [8] PyTorch Contributors. 2026. Adam. <https://docs.pytorch.org/docs/stable/generated/torch.optim.Adam.html>. Diakses 25 September 2026.
- [9] Modul Praktikum Deep Learning Minggu 3. 2026. Recurrent Neural Network untuk Pemrosesan Data Sekuensial. Program Studi Informatika, Universitas Andalas.